crisisinbox_grpo_northflank

March 8, 2026

1 CrisisInbox GRPO Training (Unsloth + OpenEnv HTTP)

Train a small LLM (Qwen2.5-0.5B) to triage crisis inbox messages using Group Relative Policy Optimization.

This notebook connects to the live CrisisInbox environment deployed on HuggingFace Spaces at `https://eptan-crisis-inbox.hf.space` via OpenEnv's WebSocket protocol.

Stack: Unsloth (2x faster LoRA) + HF TRL GRPO + OpenEnv Environment (HTTP, no MCP)

What this does: 1. Connects to the deployed CrisisInbox environment via OpenEnv WebSocket 2. Collects episodes by interacting with the environment (reset, step with tools) 3. Builds training prompts from live environment observations 4. Trains the model with GRPO using a multi-component reward function 5. Evaluates the trained model against the live environment

Open in Google Colab or Northflank with a GPU runtime.

[1]:
```
# Install dependencies: Unsloth for fast LoRA training, OpenEnv for environment
!pip install unsloth vllm -q
!pip install "openenv-core[core] @ git+https://github.com/meta-pytorch/OpenEnv.
↪git" -q
!pip install websockets huggingface_hub matplotlib -q
print("Setup complete")
```

```
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
WARNING: Ignoring invalid distribution ~orch
(/opt/conda/lib/python3.13/site-packages)
```

Setup complete

[2]:
```
# Patch transformers logging crash
import logging
import warnings

def _patch_transformers_logging():
    try:
        import transformers.utils.logging as trans_log
        _orig = trans_log.logger.warning
        def _safe_warning(msg, *args, **kwargs):
            if args and isinstance(args[0], type) and issubclass(args[0],␣
↪Warning):
                args = ()
            return _orig(msg, *args, **kwargs)
        trans_log.logger.warning = _safe_warning
    except Exception:
        pass
    warnings.filterwarnings("ignore", message=".*attention mask API.*",␣
↪category=FutureWarning)
```

```
_patch_transformers_logging()
```

[3]:
```
import torch

if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    total_bytes = getattr(props, "total_memory", None) or getattr(props,␣
 ↪"total_mem", 0)
    vram_gb = total_bytes / 1e9 if total_bytes else 0
    if vram_gb == 0 and hasattr(torch.cuda, "mem_get_info"):
        _, total_bytes = torch.cuda.mem_get_info(0)
        vram_gb = total_bytes / 1e9
    print(f"GPU: {torch.cuda.get_device_name(0)} ({vram_gb:.1f} GB)")
else:
    print("No GPU available.")
```

```
GPU: NVIDIA H100 80GB HBM3 (85.0 GB)
```

1.1 Connect to CrisisInbox Environment

Connect to the live environment on HuggingFace Spaces via OpenEnv WebSocket and collect episodes by running through the 48-hour simulation multiple times with different seeds.

[4]:
```
import json
import time as _time
from openenv import GenericEnvClient, GenericAction

BASE_URL = "https://eptan-crisis-inbox.hf.space"


def call_tool(env, tool_name, **kwargs):
    """Call a tool and return the result as a JSON string."""
    result = env.step(GenericAction(tool_name=tool_name, arguments=kwargs))
    obs = result.observation
    data = obs.get("result", obs)
    return json.dumps(data) if isinstance(data, (dict, list)) else str(data)


def step_tool(env, tool_name, **kwargs):
    """Call a tool and return the full StepResult (observation, reward, done).
 ↪"""
    return env.step(GenericAction(tool_name=tool_name, arguments=kwargs))


# Wake up the HF Space and verify connectivity
print("Connecting to HF Space (may take a moment if cold-starting)...")
for attempt in range(3):
```

```
    try:
        with GenericEnvClient(BASE_URL, message_timeout_s=60.0).sync() as env:
            r = env.reset(seed=0)
            print(f"Connected! Observation keys: {list(r.observation.keys())}")

            status = json.loads(call_tool(env, "get_status"))
            print(f"Environment ready - {status['messages_total_arrived']}␣
↪messages at hour {status['current_hour']}")
        break
    except Exception as e:
        if attempt < 2:
            print(f"  Attempt {attempt + 1} failed ({e}), retrying in 10s...")
            _time.sleep(10)
        else:
            raise RuntimeError(f"Could not connect to {BASE_URL} after 3␣
↪attempts: {e}")
```

```
Connecting to HF Space (may take a moment if cold-starting)…
Connected! Observation keys: ['result']
Environment ready - 1 messages at hour 0.0
```

[5]:
```
import random


def collect_episode(base_url, seed, time_steps=None):
    """Collect one episode from the live environment using OpenEnv␣
↪GenericEnvClient."""
    if time_steps is None:
        time_steps = list(range(0, 48, 2))  # Every 2 hours - catches nearly␣
↪all deadlines

    superseded_msgs = {}

    with GenericEnvClient(base_url, message_timeout_s=120.0).sync() as env:
        env.reset(seed=seed)
        decision_points = []
        current_hour = 0.0

        for target_hour in time_steps:
            while current_hour < target_hour - 0.1:
                jump = min(4.0, target_hour - current_hour)
                call_tool(env, "advance_time", hours=jump)
                status = json.loads(call_tool(env, "get_status"))
                current_hour = status["current_hour"]

            inbox = json.loads(call_tool(env, "get_inbox"))
            prompt = call_tool(env, "get_prompt")  # Server builds the prompt
```

```
            for m in inbox:
                if m.get("superseded"):
                    superseded_msgs[m["id"]] = ""

            unhandled = [m for m in inbox if not m.get("handled", False)]
            if not unhandled:
                continue

            decision_points.append({
                "hour": target_hour,
                "visible_count": len(inbox),
                "prompt": prompt,
                "messages": inbox,
                "superseded": dict(superseded_msgs),
            })

    return {
        "episode_id": f"ep_{seed}",
        "seed": seed,
        "drift_events": [],
        "superseded_messages": superseded_msgs,
        "decision_points": decision_points,
    }


# Test: collect one episode
print("Collecting test episode (seed=42)...")
test_ep = collect_episode(BASE_URL, seed=42)
print(f"Episode {test_ep['episode_id']}: {len(test_ep['decision_points'])}␣
 ↪decision points")
for dp in test_ep["decision_points"]:
    print(f"  Hour {dp['hour']:5.1f}: {dp['visible_count']} messages visible")
```

```
Collecting test episode (seed=42)…
Episode ep_42: 24 decision points
  Hour   0.0: 1 messages visible
  Hour   2.0: 9 messages visible
  Hour   4.0: 16 messages visible
  Hour   6.0: 22 messages visible
  Hour   8.0: 32 messages visible
  Hour  10.0: 40 messages visible
  Hour  12.0: 43 messages visible
  Hour  14.0: 48 messages visible
  Hour  16.0: 54 messages visible
  Hour  18.0: 60 messages visible
  Hour  20.0: 66 messages visible
  Hour  22.0: 68 messages visible
```

```
 Hour  24.0: 72 messages visible
 Hour  26.0: 75 messages visible
 Hour  28.0: 81 messages visible
 Hour  30.0: 82 messages visible
 Hour  32.0: 85 messages visible
 Hour  34.0: 86 messages visible
 Hour  36.0: 89 messages visible
 Hour  38.0: 91 messages visible
 Hour  40.0: 99 messages visible
 Hour  42.0: 101 messages visible
 Hour  44.0: 104 messages visible
 Hour  46.0: 105 messages visible
```

[6]:

```
# Collect multiple episodes from the live environment
NUM_EPISODES = 10
SEEDS = list(range(NUM_EPISODES))

episodes = []
for seed in SEEDS:
    print(f"Collecting episode {seed + 1}/{NUM_EPISODES} (seed={seed})...",␣
 ↪end=" ")
    for attempt in range(3):
        try:
            ep = collect_episode(BASE_URL, seed=seed)
            episodes.append(ep)
            print(f"{len(ep['decision_points'])} decision points")
            break
        except Exception as e:
            if attempt < 2:
                print(f"retry {attempt + 1}...", end=" ")
                _time.sleep(5)
            else:
                print(f"FAILED ({e}), skipping")

# Flatten to training prompts
prompts = []
for ep in episodes:
    for dp in ep["decision_points"]:
        prompts.append({
            "prompt": dp["prompt"],
            "hour": dp["hour"],
            "visible_count": dp["visible_count"],
            "episode_id": ep["episode_id"],
            "seed": ep["seed"],
            "drift_events": ep["drift_events"],
            "superseded": ep.get("superseded_messages", {}),
            "messages": dp["messages"],
```

```
        })

print(f"\nCollected {len(episodes)} episodes -> {len(prompts)} training␣
 ↪prompts")
print(f"Average {len(prompts)/len(episodes):.1f} decision points per episode")
```

```
Collecting episode 1/10 (seed=0)… 24 decision points
Collecting episode 2/10 (seed=1)… 24 decision points
Collecting episode 3/10 (seed=2)… 24 decision points
Collecting episode 4/10 (seed=3)… 24 decision points
Collecting episode 5/10 (seed=4)… 24 decision points
Collecting episode 6/10 (seed=5)… 24 decision points
Collecting episode 7/10 (seed=6)… 24 decision points
Collecting episode 8/10 (seed=7)… 24 decision points
Collecting episode 9/10 (seed=8)… 24 decision points
Collecting episode 10/10 (seed=9)… 24 decision points

Collected 10 episodes -> 240 training prompts
Average 24.0 decision points per episode
```

1.2 Reward Function (Inlined from Server)

All reward logic is inlined from `server/crisis_inbox_environment.py` — single source of truth. Training and evaluation use identical reward signals.

Components: 1. **Format reward** (0.0 to 2.0) — graduated credit for parseable output structure 2. **Urgency base** (critical=10, high=5, medium=3, low=1) 3. **Deadline timing** (early=bonus, late=penalty) 4. **Tone matching** (up to +15% for empathetic/formal responses) 5. **Drift adaptation** (+50% for handling policy-change messages) 6. **Stale info penalty** (-50% for acting on superseded messages) 7. **Conflict resolution** (+25% for handling trade-off messages) 8. **Prioritization penalty** (choosing low when critical exists)

[7]:
```
import re

# ---------------------------------------------------------------------------
# Reward function - inlined from server/rewards.py (single source of truth).
# This MUST match the server. If you change reward logic, update rewards.py
# first, then copy the changes here.
# ---------------------------------------------------------------------------

_FAMILY_SENDERS = {"mom", "sister", "neighbor dave", "emma"}
_FAMILY_TONE_WORDS = {"love", "safe", "worried", "sorry", "care", "okay",␣
 ↪"miss", "hang in there"}
_FORMAL_TONE_WORDS = {"confirm", "attached", "documentation", "regarding",␣
 ↪"request", "please", "submit"}


def tone_multiplier(sender, response):
```

```
    resp_lower = response.lower()
    sender_lower = sender.lower()
    if any(f in sender_lower for f in _FAMILY_SENDERS):
        matches = sum(1 for w in _FAMILY_TONE_WORDS if w in resp_lower)
        if matches >= 2: return 1.15
        elif matches == 1: return 1.07
    else:
        matches = sum(1 for w in _FORMAL_TONE_WORDS if w in resp_lower)
        if matches >= 2: return 1.1
        elif matches == 1: return 1.05
    return 1.0


def score_format(completion):
    """Format compliance reward (0.0 to 2.0). Bootstraps GRPO learning.

    Gives a small base reward (0.1) for any non-empty output so GRPO
    always has gradient signal, even when the model hasn't learned the format␣
↪yet.
    """
    if not completion or not completion.strip():
        return 0.0
    score = 0.1  # Base reward for non-empty output - prevents all-zero batches
    if "respond_to_message" in completion: score += 0.4
    if re.search(r'msg_\d+', completion): score += 0.5
    if re.search(r'respond_to_message\s*\(', completion): score += 0.5
    if re.search(r'respond_to_message\s*\(\s*["\']?msg_\d+["\']?\s*,\s*["\']',␣
↪completion): score += 0.5
    return score


def score_action(completion, prompt_data):
    """Score a completion: format reward (0-2) + triage reward from server␣
↪logic."""
    messages = prompt_data["messages"]
    hour = prompt_data["hour"]
    superseded = prompt_data.get("superseded", {})

    fmt_reward = score_format(completion)

    match = re.search(
        r'respond_to_message\s*\(\s*["\']?(msg_\d+)["\']?\s*,\s*["\](.+?
↪)["\]',
        completion, re.DOTALL,
    )
    if match:
        msg_id, response_text = match.group(1), match.group(2)
```

```
    else:
        id_match = re.search(r'(msg_\d+)', completion)
        if id_match:
            msg_id, response_text = id_match.group(1), completion[:200]
        else:
            return fmt_reward

    target = next((m for m in messages if m["id"] == msg_id), None)
    if target is None:
        return fmt_reward + 0.5

    # Triage reward - mirrors server/rewards.py calculate_reward()
    urgency_rewards = {"critical": 10.0, "high": 5.0, "medium": 3.0, "low": 1.0}
    triage_reward = urgency_rewards.get(target["urgency"], 1.0)

    deadline = target.get("deadline_hours")
    if deadline is not None:
        if hour <= deadline:
            triage_reward *= 1.0 + 0.5 * ((deadline - hour) / max(deadline, 1.
↪0))
        else:
            triage_reward *= 0.25

    if len(response_text.strip()) < 10:
        triage_reward *= 0.5

    triage_reward *= tone_multiplier(target.get("sender", ""), response_text)

    if target.get("drift_flag"):
        triage_reward *= 1.5

    if target["id"] in superseded:
        triage_reward *= 0.5

    if target.get("conflicts_with"):
        triage_reward *= 1.25

    unhandled = [m for m in messages if not m.get("handled") and m["id"] !=␣
↪msg_id]
    if any(m["urgency"] == "critical" for m in unhandled) and target["urgency"]␣
↪in ("low", "medium"):
        triage_reward *= 0.3

    return round(fmt_reward + triage_reward, 2)


# --- Test ---
```

```
test_data = prompts[0]
print("Testing reward function (inlined from server/rewards.py):")
print(f"  Hour: {test_data['hour']}, Messages: {test_data['visible_count']}")
print(f"\n  Format scoring:")
print(f"    Empty:              {score_format('')}:.1f} / 2.0")
print(f"    Gibberish:          {score_format('do something random')}:.1f} / 2.
↪0")
print(f"    Has msg_id:         {score_format('I should handle msg_001')}:.1f} /␣
↪2.0")
print(f"    Partial call:       {score_format('respond_to_message(msg_001')}:.
↪1f} / 2.0")
print(f"    Full call:          {score_format('respond_to_message(\"msg_001\",␣
↪\"response\")')}:.1f} / 2.0")

critical_msgs = [m for m in test_data["messages"] if m["urgency"] == "critical"]
if critical_msgs:
    good = f'respond_to_message("{critical_msgs[0]["id"]}", "Evacuating now␣
↪with documents.")'
    print(f"\n  Full reward (critical, good response): {score_action(good,␣
↪test_data)}:.2f}")
low_msgs = [m for m in test_data["messages"] if m["urgency"] == "low"]
if low_msgs:
    bad = f'respond_to_message("{low_msgs[0]["id"]}", "ok")'
    print(f"  Full reward (low, short response):     {score_action(bad,␣
↪test_data)}:.2f}")
print(f"  Unparseable (format only):             {score_action('do something',␣
↪test_data)}:.2f}")
```

```
Testing reward function (inlined from server/rewards.py):
  Hour: 0, Messages: 1

  Format scoring:
    Empty:              0.0 / 2.0
    Gibberish:          0.1 / 2.0
    Has msg_id:         0.6 / 2.0
    Partial call:       1.5 / 2.0
    Full call:          2.0 / 2.0

  Full reward (critical, good response): 17.00
  Unparseable (format only):             0.10
```

1.3 Load Model & Baseline Evaluation

Load the model, run a **pre-training baseline** against the live environment, then configure GRPO training.

[8]:
```
from unsloth import FastLanguageModel
import torch

max_seq_length = 2048  # Qwen2.5-0.5B supports 32K; need room for prompt (1024)␣
↪+ completion (256)

# Unsloth handles quantization, dtype selection, and device mapping␣
↪automatically
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="Qwen/Qwen2.5-0.5B-Instruct",
    max_seq_length=max_seq_length,
    load_in_4bit=True,
)

# Apply LoRA adapters via Unsloth (patches for 2x speed)
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.0,
    bias="none",
)

# Auto-detect precision
_use_bf16 = torch.cuda.is_bf16_supported() if torch.cuda.is_available() else␣
↪False

# GRPO requires left padding so completions align across the batch
tokenizer.padding_side = "left"
if tokenizer.pad_token_id is None:
    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.pad_token_id = tokenizer.eos_token_id

print(f"Model loaded with Unsloth (4-bit quantized,␣
↪max_seq_length={max_seq_length})")
print(f"Precision: {'bf16' if _use_bf16 else 'fp16'}")
print(f"Trainable params: {sum(p.numel() for p in model.parameters() if p.
↪requires_grad) / 1e6:.1f}M")
```

```
/tmp/ipykernel_28467/2226621945.py:1: UserWarning: WARNING: Unsloth should be
imported before [transformers] to ensure all optimizations are applied. Your
code may run slower or encounter memory issues without these optimizations.

Please restructure your imports with 'import unsloth' at the top of your file.
  from unsloth import FastLanguageModel
```

```
 Unsloth: Will patch your computer to enable 2x faster free finetuning.

/opt/conda/lib/python3.13/site-packages/torch/jit/_script.py:362:
DeprecationWarning: `torch.jit.script_method` is deprecated. Please switch to
`torch.compile` or `torch.export`.
  warnings.warn(

 Unsloth Zoo will now patch everything to make training faster!
==((====))==  Unsloth 2026.3.3: Fast Qwen2 patching. Transformers: 4.57.6. vLLM:
0.17.0.
   \\   /|    NVIDIA H100 80GB HBM3. Num GPUs = 1. Max memory: 79.179 GB.
Platform: Linux.
O^O/ \_/ \    Torch: 2.10.0+cu128. CUDA: 9.0. CUDA Toolkit: 12.8. Triton: 3.6.0
\        /    Bfloat16 = TRUE. FA [Xformers = 0.0.35. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red
colored!
unsloth/qwen2.5-0.5b-instruct-unsloth-bnb-4bit does not have a padding token!
Will use pad_token = <|PAD_TOKEN|>.

Unsloth 2026.3.3 patched 24 layers with 24 QKV layers, 24 O layers and 24 MLP
layers.

Model loaded with Unsloth (4-bit quantized, max_seq_length=2048)
Precision: bf16
Trainable params: 8.8M
```

1.3.1 Pre-Training Baseline

Evaluate the **untrained** model against the live environment before any GRPO training. This gives us a baseline to measure improvement.

[9]:
```
# --- Shared utilities (used by both eval and dataset prep) ---

MAX_PROMPT_LENGTH = max_seq_length - 512  # 2048 - 512 = 1536 tokens (leaves␣
↪room for completion + chat template overhead)
TIME_STEPS = list(range(0, 48, 2))  # Same as collect_episode


def truncate_prompt_to_fit(prompt_text, tokenizer, max_tokens):
    """Truncate prompt text so it tokenizes to <= max_tokens.

    Strategy: tokenize, truncate tokens from the LEFT (drop oldest messages,
    keep recent context), then decode back to text.
    """
    msgs = [{"role": "user", "content": prompt_text}]
    ids = tokenizer.apply_chat_template(msgs, return_tensors="pt",␣
↪add_generation_prompt=True)
    if hasattr(ids, "input_ids"):
        ids = ids.input_ids
```

```
    elif isinstance(ids, dict):
        ids = ids["input_ids"]
    if not isinstance(ids, torch.Tensor):
        ids = torch.tensor(ids) if isinstance(ids, list) else ids
    if ids.dim() > 1:
        ids = ids[0]

    n_tokens = ids.shape[0]
    if n_tokens <= max_tokens:
        return prompt_text  # Already fits

    # Truncate from left (keep recent messages), then decode back
    truncated_ids = ids[-max_tokens:]
    truncated_text = tokenizer.decode(truncated_ids, skip_special_tokens=True)
    return truncated_text


def generate_action(model, tokenizer, prompt_text):
    """Generate an action from the model, truncating prompt the same way as␣
 ↪training."""
    FastLanguageModel.for_inference(model)
    prompt_text = truncate_prompt_to_fit(prompt_text, tokenizer,␣
 ↪MAX_PROMPT_LENGTH)
    msgs = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": prompt_text},
    ]
    input_ids = tokenizer.apply_chat_template(msgs, return_tensors="pt",␣
 ↪add_generation_prompt=True)
    if hasattr(input_ids, "input_ids"):
        input_ids = input_ids.input_ids
    elif isinstance(input_ids, dict):
        input_ids = input_ids["input_ids"]
    if not isinstance(input_ids, torch.Tensor):
        input_ids = torch.tensor(input_ids)
    input_ids = input_ids.to("cuda")
    prompt_len = input_ids.shape[-1] if input_ids.dim() > 1 else input_ids.
 ↪shape[0]
    with torch.no_grad():
        output = model.generate(input_ids=input_ids, max_new_tokens=256,␣
 ↪temperature=0.7,
                                pad_token_id=tokenizer.pad_token_id,␣
 ↪do_sample=True)
    return tokenizer.decode(output[0][prompt_len:], skip_special_tokens=True)
```

```python
def score_action_detailed(completion, prompt_data):
    """Like score_action but returns (total, format_reward, triage_reward)␣
↪breakdown."""
    fmt = score_format(completion)
    total = score_action(completion, prompt_data)
    triage = round(total - fmt, 2)
    return total, fmt, triage


def evaluate_on_live_env(model, tokenizer, base_url, seed, time_steps=None):
    """Evaluate model against the live environment via OpenEnv GenericEnvClient.
↪"""
    if time_steps is None:
        time_steps = TIME_STEPS

    with GenericEnvClient(base_url, message_timeout_s=120.0).sync() as env:
        env.reset(seed=seed)
        total_reward = 0.0
        total_format = 0.0
        total_triage = 0.0
        actions_taken = []
        current_hour = 0.0

        for target_hour in time_steps:
            while current_hour < target_hour - 0.1:
                jump = min(4.0, target_hour - current_hour)
                call_tool(env, "advance_time", hours=jump)
                status = json.loads(call_tool(env, "get_status"))
                current_hour = status["current_hour"]

            inbox = json.loads(call_tool(env, "get_inbox"))
            unhandled = [m for m in inbox if not m.get("handled", False)]

            if not unhandled:
                continue

            prompt = call_tool(env, "get_prompt")
            completion = generate_action(model, tokenizer, prompt)

            match = re.search(r'respond_to_message\s*\(\s*["\']?(msg_\d+)["\']?
↪\s*,\s*["\'](.+?)["\']', completion, re.DOTALL)
            if not match:
                id_match = re.search(r'(msg_\d+)', completion)
                if id_match:
                    msg_id, response_text = id_match.group(1), completion[:200]
                else:
                    continue
```

```
            else:
                msg_id, response_text = match.group(1), match.group(2)

            result = step_tool(env, "respond_to_message", message_id=msg_id,␣
↪response=response_text)
            obs = result.observation
            result_data = obs.get("result", {})
            if isinstance(result_data, str):
                result_data = json.loads(result_data)

            if "error" in result_data:
                continue

            prompt_data = {"messages": inbox, "hour": current_hour,␣
↪"superseded": {
                m["id"]: "" for m in inbox if m.get("superseded", False)
            }}
            reward, fmt_r, triage_r = score_action_detailed(completion,␣
↪prompt_data)

            total_reward += reward
            total_format += fmt_r
            total_triage += triage_r
            target_msg = next((m for m in inbox if m["id"] == msg_id), None)
            urgency = target_msg["urgency"] if target_msg else "?"
            actions_taken.append({"step": len(actions_taken), "hour":␣
↪current_hour, "msg_id": msg_id, "urgency": urgency,
                                  "reward": reward, "format": fmt_r, "triage":␣
↪triage_r})
            print(f"  Step {len(actions_taken)-1:2d} | Hour {current_hour:5.1f}␣
↪| {msg_id} ({urgency:8s}) | "
                  f"Fmt: {fmt_r:.1f} + Triage: {triage_r:+.1f} = {reward:+.1f}␣
↪| Total: {total_reward:.1f}")

        final_status = json.loads(call_tool(env, "get_status"))
        print(f"  {'-'*70}")
        print(f"  Totals: Format={total_format:.1f}  Triage={total_triage:.1f} ␣
↪Combined={total_reward:.1f}")

    return {"seed": seed, "total_reward": total_reward, "total_format":␣
↪total_format,
            "total_triage": total_triage, "actions": actions_taken,␣
↪"final_status": final_status}


print(f"Utilities ready. MAX_PROMPT_LENGTH={MAX_PROMPT_LENGTH} tokens")
```

```
Utilities ready. MAX_PROMPT_LENGTH=1536 tokens
```

[10]:
```python
from datasets import Dataset

# MAX_PROMPT_LENGTH and truncate_prompt_to_fit defined above (shared with eval)

SYSTEM_PROMPT = """You are a crisis inbox triage assistant. You must respond␣
↪using EXACTLY this format:

respond_to_message("msg_XXX", "your response here")

Example:
respond_to_message("msg_001", "Evacuating now. Bringing documents and␣
↪medication.")

Rules:
- Output ONLY the respond_to_message() call, nothing else
- Pick the most urgent unhandled message (critical > high > medium > low)
- Watch deadlines - respond before they expire
- Adapt to policy changes (schema drift)"""

train_data = []
prompt_lookup = {}  # Build lookup HERE using truncated text as key (reward_fn␣
↪sees truncated text)
truncated_count = 0
for p in prompts:
    prompt_text = p["prompt"]

    # Truncate to fit (same function used by generate_action during eval)
    truncated = truncate_prompt_to_fit(prompt_text, tokenizer,␣
↪MAX_PROMPT_LENGTH)
    if truncated != prompt_text:
        truncated_count += 1

    # Key by truncated text (this is what reward_fn will see during training)
    key = truncated[:200]
    prompt_lookup[key] = p

    msgs = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": truncated},
    ]
    train_data.append({
        "prompt": msgs,
    })

random.seed(42)
```

```
random.shuffle(train_data)

dataset = Dataset.from_list(train_data)
print(f"Training dataset: {len(dataset)} prompts ({truncated_count} truncated␣
↪to fit {MAX_PROMPT_LENGTH} tokens)")
print(f"Prompt lookup: {len(prompt_lookup)} keys (keyed by truncated text)")
print(f"Sample prompt length: {len(train_data[0]['prompt'][1]['content'])}␣
↪chars")
print(f"System prompt: {len(SYSTEM_PROMPT)} chars")
```

```
Training dataset: 240 prompts (180 truncated to fit 1536 tokens)
Prompt lookup: 197 keys (keyed by truncated text)
Sample prompt length: 5006 chars
System prompt: 459 chars
```

1.4 GRPO Training Loop

The reward function scores each completion by: 1. Parsing which message the model chose to handle 2. Checking urgency, deadline timing, drift flags 3. Penalizing bad choices (low-urgency when critical exists, stale info)

[11]:
```
import gc
gc.collect()
torch.cuda.empty_cache()
```

[12]:
```
from trl import GRPOConfig, GRPOTrainer

# prompt_lookup is built in the dataset cell above (keyed by truncated text)
# We use it to get the hour/seed for each prompt so we can reset the env to the␣
↪right state

_reward_hit = 0
_reward_miss = 0
_env_calls = 0


def reward_fn(prompts, completions, **kwargs):
    """Online GRPO reward function - executes actions via OpenEnv env.step().

    For each completion:
    1. Parse the respond_to_message() call
    2. Reset environment to the correct episode state (seed + hour)
    3. Call env.step(GenericAction) with respond_to_message
    4. Return the environment's reward

    Falls back to local scoring if the environment is unreachable.
    """
    global _reward_hit, _reward_miss, _env_calls
```

```
    rewards = []

    for prompt_msgs, completion in zip(prompts, completions):
        if isinstance(prompt_msgs, list):
            prompt_text = prompt_msgs[-1]["content"] if prompt_msgs else ""
        else:
            prompt_text = str(prompt_msgs)

        key = prompt_text[:200]
        prompt_data = prompt_lookup.get(key)

        if isinstance(completion, list):
            if completion and isinstance(completion[0], (int, float)):
                comp_text = tokenizer.decode(completion,␣
↪skip_special_tokens=True)
            else:
                comp_text = completion[-1].get("content", "") if completion␣
↪else ""
        else:
            comp_text = str(completion)

        if prompt_data is None:
            _reward_miss += 1
            rewards.append(0.0)
            continue

        _reward_hit += 1

        # Parse the completion to extract action
        match = re.search(
            r'respond_to_message\s*\(\s*["\']?(msg_\d+)["\']?\s*,\s*["\](.+?
↪)["\']',
            comp_text, re.DOTALL,
        )
        if not match:
            id_match = re.search(r'(msg_\d+)', comp_text)
            if id_match:
                msg_id, response_text = id_match.group(1), comp_text[:200]
            else:
                # No parseable action - return format-only reward
                rewards.append(score_format(comp_text))
                continue
        else:
            msg_id, response_text = match.group(1), match.group(2)

        # Execute via OpenEnv env.step()
        try:
```

```
            seed = prompt_data.get("seed", 0)
            hour = prompt_data.get("hour", 0)

            with GenericEnvClient(BASE_URL, message_timeout_s=30.0).sync() as␣
↪env:
                env.reset(seed=seed)
                # Advance to the correct hour
                remaining = hour
                while remaining > 0.1:
                    call_tool(env, "advance_time", hours=min(remaining, 4.0))
                    remaining -= 4.0

                # Step through the environment with the model's action
                result = env.step(GenericAction(
                    tool_name="respond_to_message",
                    arguments={"message_id": msg_id, "response": response_text}
                ))

                # Environment returns reward directly
                env_reward = result.reward if result.reward else 0.0

                # Add format reward on top (environment doesn't score format)
                rewards.append(score_format(comp_text) + float(env_reward))
                _env_calls += 1

        except Exception:
            # Fallback to local scoring if env unreachable
            rewards.append(score_action(comp_text, prompt_data))

    return rewards


training_args = GRPOConfig(
    output_dir="crisisinbox-grpo-output",
    max_steps=200,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=1,
    learning_rate=1e-6,
    max_completion_length=256,
    max_prompt_length=MAX_PROMPT_LENGTH,  # 1536 - fits prompt + completion +␣
↪chat template within 2048
    num_generations=16,
    generation_batch_size=16,
    logging_steps=8,  # Log every generation batch (16 gens / 2 batch_size = 8␣
↪steps) - no blank rows
    save_steps=200,
    bf16=_use_bf16,
```

```
    fp16=not _use_bf16,
    sync_ref_model=True,
)

# Unsloth: LoRA is already applied to the model, so no peft_config needed
trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=reward_fn,
    args=training_args,
    train_dataset=dataset,
)

model = trainer.model

print(f"Trainer configured - {len(prompt_lookup)} unique prompt keys")
print(f"Rewards: ONLINE via OpenEnv env.step() → respond_to_message (live at␣
↪{BASE_URL})")
print(f"Fallback: local score_action if environment unreachable")
print(f"max_seq_length={max_seq_length}, max_prompt_length={MAX_PROMPT_LENGTH},␣
↪max_completion_length=256")
print(f"Precision: {'bf16' if _use_bf16 else 'fp16'}")
print(f"LR: {training_args.learning_rate}, Max steps: {training_args.
↪max_steps}, Generations: {training_args.num_generations}")
print(f"Logging every {training_args.logging_steps} steps (1 row per generation␣
↪batch)")
print("Ready to train (Unsloth 2x speedup, online rewards via OpenEnv env.
↪step())")
```

```
Trainer configured - 197 unique prompt keys
Rewards: ONLINE via OpenEnv env.step() → respond_to_message (live at
https://eptan-crisis-inbox.hf.space)
Fallback: local score_action if environment unreachable
max_seq_length=2048, max_prompt_length=1536, max_completion_length=256
Precision: bf16
LR: 1e-06, Max steps: 200, Generations: 16
Logging every 8 steps (1 row per generation batch)
Ready to train (Unsloth 2x speedup, online rewards via OpenEnv env.step())
```

[13]:
```
# Train with online rewards from OpenEnv!
trainer.train()
print("Training complete")
print(f"Reward lookup stats: {_reward_hit} hits, {_reward_miss} misses␣
↪({_reward_hit/max(_reward_hit+_reward_miss,1)*100:.0f}% hit rate)")
print(f"OpenEnv environment calls: {_env_calls} (online scoring via␣
↪score_completions)")
print(f"Local fallback calls: {_reward_hit - _env_calls}")
```

```
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 240 | Num Epochs = 1 | Total steps = 200
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 1
\        /    Data Parallel GPUs = 1 | Total batch size (2 x 1 x 1) = 2
 "-____-"     Trainable parameters = 8,798,208 of 502,830,976 (1.75% trained)
```

```
Unsloth: Will smartly offload gradients to save VRAM!
```

```
<IPython.core.display.HTML object>
```

```
Training complete
Reward lookup stats: 400 hits, 0 misses (100% hit rate)
OpenEnv environment calls: 309 (online scoring via score_completions)
Local fallback calls: 91
```

1.5 Evaluate: Offline + Training Curve

Evaluate the trained model on collected prompts and plot the training reward curve.

[14]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

model.eval()

# --- Post-training evaluation ---
print("=== POST-TRAINING EVALUATION ===\n")
trained_results = []
for seed in [99, 42, 7]:
    print(f"--- Seed {seed} ---")
    res = evaluate_on_live_env(model, tokenizer, BASE_URL, seed=seed)
    trained_results.append(res)
    print(f"  Total: {res['total_reward']:.1f} | Actions:␣
 ↪{len(res['actions'])}\n")

trained_avg = sum(r["total_reward"] for r in trained_results) /␣
 ↪len(trained_results)
print(f"Trained average reward: {trained_avg:.1f}")

# --- Training reward curve with trend line ---
history = pd.DataFrame(trainer.state.log_history)

fig, ax = plt.subplots(figsize=(10, 5))

if "rewards/reward_fn/mean" in history.columns:
    reward_steps = history.dropna(subset=["rewards/reward_fn/mean"])
    steps = reward_steps["step"].values
    means = reward_steps["rewards/reward_fn/mean"].values
    stds = reward_steps["rewards/reward_fn/std"].values
```

```
    # Raw reward curve
    ax.plot(steps, means, label="Mean Reward", color="#2ca02c", linewidth=1.5,␣
↪alpha=0.6)
    ax.fill_between(steps, means - stds, means + stds, alpha=0.15,␣
↪color="#2ca02c")

    # Trend line (polynomial fit)
    if len(steps) >= 3:
        z = np.polyfit(steps, means, 2)  # Quadratic trend
        trend = np.polyval(z, steps)
        ax.plot(steps, trend, label="Trend", color="#d62728", linewidth=2.5,␣
↪linestyle="--")

    ax.set_xlabel("Training Steps")
    ax.set_ylabel("Reward (per action)")
    ax.set_title(f"GRPO Training Curve - Post-Training Eval Avg: {trained_avg:.
↪1f}")
    ax.legend()
    ax.grid(True, linestyle="--", alpha=0.6)
else:
    ax.text(0.5, 0.5, "No reward history", ha="center", va="center",
            transform=ax.transAxes, fontsize=12)
    ax.set_title("GRPO Training Curve")

plt.tight_layout()
plt.savefig("crisisinbox_grpo_results.png", dpi=150, bbox_inches="tight")
plt.show()
print("Results saved to crisisinbox_grpo_results.png")
```

```
=== POST-TRAINING EVALUATION ===

--- Seed 99 ---
  Step  0 | Hour   0.0 | msg_001 (critical) | Fmt: 2.0 + Triage: +15.0 = +17.0 |
Total: 17.0
  Step  1 | Hour   2.2 | msg_086 (medium  ) | Fmt: 0.6 + Triage: +0.9 = +1.5 |
Total: 18.5
  Step  2 | Hour   4.2 | msg_078 (medium  ) | Fmt: 2.0 + Triage: +1.0 = +3.0 |
Total: 21.5
  Step  3 | Hour   6.2 | msg_084 (critical) | Fmt: 1.5 + Triage: +2.5 = +4.0 |
Total: 25.5
  Step  4 | Hour   8.2 | msg_002 (high    ) | Fmt: 2.0 + Triage: +5.3 = +7.3 |
Total: 32.9
  Step  5 | Hour  14.0 | msg_083 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
Total: 35.8
  Step  6 | Hour  20.0 | msg_031 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 38.1
```

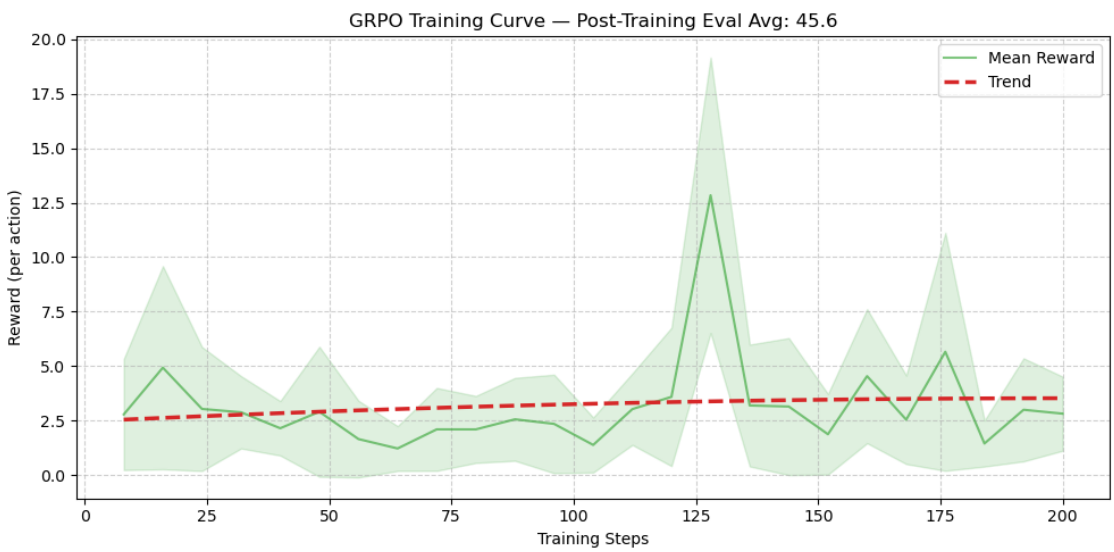
```
  Step  7 | Hour  32.0 | msg_048 (medium  ) | Fmt: 2.0 + Triage: +0.5 = +2.5 |
Total: 40.5
  Step  8 | Hour  36.0 | msg_100 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 42.8
  Step  9 | Hour  40.0 | msg_079 (high    ) | Fmt: 2.0 + Triage: +0.6 = +2.6 |
Total: 45.4
  Step 10 | Hour  46.0 | msg_072 (low     ) | Fmt: 2.0 + Triage: +0.1 = +2.1 |
Total: 47.6
  ----------------------------------------------------------------------
  Totals: Format=20.1  Triage=27.5  Combined=47.6
  Total: 47.6 | Actions: 11

--- Seed 42 ---
  Step  0 | Hour   0.0 | msg_001 (critical) | Fmt: 0.6 + Triage: +15.0 = +15.6 |
Total: 15.6
  Step  1 | Hour   2.2 | msg_086 (medium  ) | Fmt: 0.6 + Triage: +0.9 = +1.5 |
Total: 17.1
  Step  2 | Hour   4.2 | msg_008 (medium  ) | Fmt: 2.0 + Triage: +1.3 = +3.3 |
Total: 20.4
  Step  3 | Hour   6.2 | msg_002 (high    ) | Fmt: 2.0 + Triage: +5.3 = +7.3 |
Total: 27.8
  Step  4 | Hour  20.0 | msg_079 (high    ) | Fmt: 1.5 + Triage: +1.2 = +2.8 |
Total: 30.5
  Step  5 | Hour  24.0 | msg_019 (low     ) | Fmt: 2.0 + Triage: +0.1 = +2.1 |
Total: 32.7
  Step  6 | Hour  26.2 | msg_077 (high    ) | Fmt: 2.0 + Triage: +0.8 = +2.8 |
Total: 35.4
  Step  7 | Hour  28.2 | msg_100 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 37.7
  Step  8 | Hour  30.2 | msg_031 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 40.0
  Step  9 | Hour  36.0 | msg_023 (medium  ) | Fmt: 2.0 + Triage: +0.5 = +2.5 |
Total: 42.5
  Step 10 | Hour  42.0 | msg_103 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 44.8
  Step 11 | Hour  46.0 | msg_101 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
Total: 47.7
  ----------------------------------------------------------------------
  Totals: Format=20.7  Triage=27.0  Combined=47.7
  Total: 47.7 | Actions: 12

--- Seed 7 ---
  Step  0 | Hour   0.0 | msg_001 (critical) | Fmt: 0.6 + Triage: +15.0 = +15.6 |
Total: 15.6
  Step  1 | Hour   2.2 | msg_002 (high    ) | Fmt: 0.6 + Triage: +5.3 = +6.0 |
Total: 21.6
  Step  2 | Hour   4.2 | msg_008 (medium  ) | Fmt: 2.0 + Triage: +0.7 = +2.7 |
Total: 24.2
```

```
  Step  3 | Hour  14.0 | msg_007 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
Total: 27.1
  Step  4 | Hour  18.0 | msg_079 (high    ) | Fmt: 1.5 + Triage: +1.2 = +2.8 |
Total: 29.9
  Step  5 | Hour  22.0 | msg_034 (high    ) | Fmt: 2.0 + Triage: +2.5 = +4.5 |
Total: 34.4
  Step  6 | Hour  30.0 | msg_051 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
Total: 37.3
  Step  7 | Hour  34.0 | msg_078 (medium  ) | Fmt: 1.5 + Triage: +0.2 = +1.7 |
Total: 39.0
  Step  8 | Hour  40.0 | msg_102 (low     ) | Fmt: 0.6 + Triage: +0.3 = +0.9 |
Total: 39.9
  Step  9 | Hour  46.0 | msg_071 (medium  ) | Fmt: 0.6 + Triage: +0.9 = +1.5 |
Total: 41.4
  ----------------------------------------------------------------------
  Totals: Format=13.4  Triage=28.0  Combined=41.4
  Total: 41.4 | Actions: 10

Trained average reward: 45.6
```




```
Results saved to crisisinbox_grpo_results.png
```

1.6 Evaluate Against Live Environment

Run the trained model in a closed loop against the actual CrisisInbox environment to get real server-side rewards.

[15]:
```
# === Schema Drift Impact Visualization ===
# Show how the agent handles policy changes mid-episode
```

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# Known drift event trigger hours (from drift_events.py)
DRIFT_EVENTS = {
    20.0: "Insurance deadline\nshortened (72h→48h)",
    16.0: "Evacuation zone\nexpands to workplace",
    24.0: "Employer switches\nto paid emergency leave",
    12.0: "Airline extends\nfree rebooking (48h→7d)",
    28.0: "FEMA adds new\ndocumentation requirements",
    18.0: "Shelter closes\n(structural damage)",
    30.0: "Fuel rationing\n(odd/even plates)",
    22.0: "Pharmacy closures\n(prescription transfers)",
    32.0: "Curfew extended\n(6PM-8AM)",
}

# Run a detailed eval to capture per-action timing
print("=== Schema Drift Impact Analysis (seed=42) ===\n")
drift_eval = evaluate_on_live_env(model, tokenizer, BASE_URL, seed=42)

# Classify actions as pre-drift vs post-drift (drift events fire at hours 12-32)
actions = drift_eval["actions"]
pre_drift = [a for a in actions if a["hour"] < 12]
during_drift = [a for a in actions if 12 <= a["hour"] <= 34]
post_drift = [a for a in actions if a["hour"] > 34]

pre_avg = sum(a["reward"] for a in pre_drift) / max(len(pre_drift), 1)
during_avg = sum(a["reward"] for a in during_drift) / max(len(during_drift), 1)
post_avg = sum(a["reward"] for a in post_drift) / max(len(post_drift), 1)

print(f"\n  Pre-drift  (hour 0-12):  {len(pre_drift):2d} actions, avg reward␣
 ↪{pre_avg:.1f}")
print(f"  During drift (12-34):    {len(during_drift):2d} actions, avg reward␣
 ↪{during_avg:.1f}")
print(f"  Post-drift  (hour 34+):  {len(post_drift):2d} actions, avg reward␣
 ↪{post_avg:.1f}")

# --- Timeline plot ---
fig, ax = plt.subplots(figsize=(14, 5))

# Background shading for drift window
ax.axvspan(12, 34, alpha=0.1, color="red", label="Schema Drift Window")

# Plot each action as a point
hours = [a["hour"] for a in actions]
rewards = [a["reward"] for a in actions]
```

```
urgencies = [a["urgency"] for a in actions]
colors = {"critical": "#d62728", "high": "#ff7f0e", "medium": "#2ca02c", "low":␣
↪"#1f77b4"}
point_colors = [colors.get(u, "#999999") for u in urgencies]

ax.scatter(hours, rewards, c=point_colors, s=80, zorder=5, edgecolors="white",␣
↪linewidth=0.5)

# Mark drift event times
for hour, label in DRIFT_EVENTS.items():
    ax.axvline(x=hour, color="#d62728", alpha=0.3, linestyle=":", linewidth=1)

# Add drift event annotations (only a few to avoid clutter)
drift_subset = {16.0: "Evac zone expands", 20.0: "Insurance 72h→48h", 24.0:␣
↪"Paid leave activated"}
for hour, label in drift_subset.items():
    ax.annotate(label, xy=(hour, max(rewards) * 0.95), fontsize=7,␣
↪color="#d62728",
                rotation=90, ha="right", va="top", alpha=0.7)

# Urgency legend
legend_handles = [mpatches.Patch(color=c, label=u.title()) for u, c in colors.
↪items()]
legend_handles.append(mpatches.Patch(color="red", alpha=0.1, label="Drift␣
↪Window"))
ax.legend(handles=legend_handles, loc="upper left", fontsize=8)

ax.set_xlabel("Episode Hour (48h timeline)")
ax.set_ylabel("Reward per Action")
ax.set_title("Agent Triage Timeline with Schema Drift Events")
ax.set_xlim(0, 48)
ax.grid(True, linestyle="--", alpha=0.4)

plt.tight_layout()
plt.savefig("crisisinbox_drift_impact.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved to crisisinbox_drift_impact.png")
```

```
=== Schema Drift Impact Analysis (seed=42) ===

  Step  0 | Hour   0.0 | msg_001 (critical) | Fmt: 0.6 + Triage: +15.0 = +15.6 |
Total: 15.6
  Step  1 | Hour   2.2 | msg_086 (medium  ) | Fmt: 0.6 + Triage: +0.9 = +1.5 |
Total: 17.1
  Step  2 | Hour   8.0 | msg_080 (critical) | Fmt: 2.0 + Triage: +2.5 = +4.5 |
Total: 21.6
  Step  3 | Hour  10.2 | msg_083 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
```

```
Total: 24.5
  Step  4 | Hour  12.2 | msg_018 (medium  ) | Fmt: 2.0 + Triage: +0.9 = +2.9 |
Total: 27.4
  Step  5 | Hour  14.2 | msg_020 (high    ) | Fmt: 2.0 + Triage: +5.0 = +7.0 |
Total: 34.4
  Step  6 | Hour  18.0 | msg_031 (low     ) | Fmt: 2.0 + Triage: +0.1 = +2.1 |
Total: 36.5
  Step  7 | Hour  20.2 | msg_082 (high    ) | Fmt: 2.0 + Triage: +1.2 = +3.2 |
Total: 39.8
  Step  8 | Hour  22.2 | msg_029 (critical) | Fmt: 0.6 + Triage: +10.0 = +10.6 |
Total: 50.4
  Step  9 | Hour  24.2 | msg_033 (medium  ) | Fmt: 2.0 + Triage: +0.5 = +2.5 |
Total: 52.9
  Step 10 | Hour  26.2 | msg_037 (critical) | Fmt: 0.6 + Triage: +10.1 = +10.7 |
Total: 63.6
  Step 11 | Hour  28.2 | msg_076 (high    ) | Fmt: 0.6 + Triage: +1.6 = +2.2 |
Total: 65.7
  Step 12 | Hour  30.2 | msg_021 (low     ) | Fmt: 2.0 + Triage: +0.3 = +2.3 |
Total: 68.0
  Step 13 | Hour  32.2 | msg_022 (low     ) | Fmt: 2.0 + Triage: +0.1 = +2.1 |
Total: 70.2
  Step 14 | Hour  34.2 | msg_030 (high    ) | Fmt: 2.0 + Triage: +0.6 = +2.6 |
Total: 72.8
  Step 15 | Hour  36.2 | msg_023 (medium  ) | Fmt: 2.0 + Triage: +0.5 = +2.5 |
Total: 75.3
  Step 16 | Hour  46.0 | msg_046 (medium  ) | Fmt: 2.0 + Triage: +0.5 = +2.5 |
Total: 77.7
  ----------------------------------------------------------------------
  Totals: Format=27.0  Triage=50.7  Combined=77.7

  Pre-drift  (hour 0-12):   4 actions, avg reward 6.1
  During drift (12-34):    10 actions, avg reward 4.6
  Post-drift  (hour 34+):   3 actions, avg reward 2.5
```

```
Saved to crisisinbox_drift_impact.png
```

[16]:
```python
# Save the trained LoRA adapter
model.save_pretrained("crisisinbox-grpo-trained")
tokenizer.save_pretrained("crisisinbox-grpo-trained")
print("LoRA adapter saved to crisisinbox-grpo-trained/")
```

```
LoRA adapter saved to crisisinbox-grpo-trained/
```